Divide-and-Conquer in Practice: Closest Pair of Points and Karatsuba Integer Multiplication

CS302 Design and Analysis of Algorithms — Fall 2026 — Project Report
Talal Ali (24K-0863), <Member 2 (roll no.)>, <Member 3 (roll no.)> — FAST-NUCES Karachi

Abstract

This project implements two classical divide-and-conquer algorithms — the Θ(n log n) closest-pair-of-points algorithm and Karatsuba's Θ(n^1.585) integer multiplication — and packages them in a system that generates random test data, benchmarks the algorithms against their brute-force counterparts, and visualises every divide and conquer step in an interactive browser interface. Twenty input files (150–80,000 points; 128–65,536-digit integers) were generated with a fixed seed. On every file both algorithms produced results identical to the brute-force or built-in reference, and the expected asymptotic gap was observed: closest pair ran 92× faster than brute force at n = 5,000, and Karatsuba overtook schoolbook multiplication at about 4,000 digits, reaching 3.6× at 16,384 digits.

1. Introduction

Divide and conquer solves a problem by splitting it into smaller instances of itself, solving those recursively, and combining the answers [1]. Its power comes from the combine step: if combining costs less than solving from scratch, the Master Theorem turns a quadratic brute-force bound into n log n or n^1.585. The closest-pair problem (find the two nearest among n points in the plane) has an obvious $\Theta(n^2)$ brute-force solution; the divide-and-conquer version of Shamos and Hoey sorts the points, splits at the median x, and resolves the cross-boundary case by scanning a narrow strip in y-order where each point has at most seven relevant neighbours [1, 2]. Karatsuba's 1960 algorithm multiplies two n-digit integers with three recursive half-size products instead of four by using (a+b)(c+d) − ac − bd = ad + bc, giving $M(n) = 3M(n/2) + \Theta(n) \in \Theta(n^{\log_2 3})$ [1, 3]. The objectives of this project were to (i) implement both algorithms, (ii) generate and store random datasets of increasing size, (iii) benchmark them against brute force and verify correctness, and (iv) build a user interface that lets a user pick any input file and watch the algorithm work.

2. Proposed System

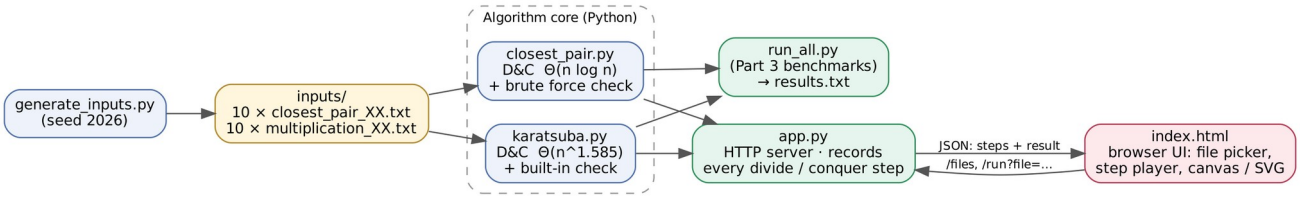


Figure 1. System architecture: data generation → algorithm core → benchmark runner and visual interface.

The system (Figure 1) is written entirely in Python 3 with no third-party packages, plus one HTML/JavaScript page. **generate_inputs.py** writes ten point files and ten integer files with a fixed random seed so every run is reproducible. **closest_pair.py** implements the recursive algorithm: points are pre-sorted by x and by y, each call splits at the median x, filters the y-sorted list into the two halves in linear time, and after the recursive calls scans the strip of half-width d = min(d_L, d_R), comparing each strip point only with the following points whose y-gap is below d. **karatsuba.py** splits each operand at h = ⌊n/2⌋ digits with divmod, performs the three recursive products, and recombines with shifts; the recursion stops at single digits. **run_all.py** (Part 3) runs every file, times the divide-and-conquer and brute-force versions, asserts that they agree, and writes results.txt. **app.py** (Part 4) is a standard-library HTTP server that runs the same algorithm code with instrumentation: every divide step records the dividing line and region, every conquer step records the strip, the number of pairs checked and the best pair; for Karatsuba the recursion tree is recorded to depth 3 with the actual values of a, b, c, d, ac, bd and (a+b)(c+d). The browser page (**index.html**) lists all input files, calls /run on the chosen one and plays the recorded steps on an HTML canvas (points, lines, strip, best pair) or an SVG recursion tree, with play/pause, single-step and speed controls, while a side panel explains the current step with its real numbers and the final result with timings and the brute-force cross-check.

3. Experimental Setup

All experiments ran on a single core with CPython 3.13 (Linux); times are wall-clock milliseconds from time.perf_counter. Inputs were produced by generate_inputs.py with random.seed(2026). **Closest pair:** file i holds n_i points with coordinates drawn uniformly from [0, 10000] with three decimals, n = 150, 300, 600, 1200, 2500, 5000, 10000, 20000, 40000, 80000 (first line n, then one “x y” pair per line). The first points of closest_pair_01 are (1191.199, 5025.158), (5118.227, 8600.006); of closest_pair_05 (3408.303, 9701.803), (2367.548, 2261.091); of closest_pair_10 (2266.877, 5063.293), (7750.878, 1197.135). **Integer multiplication:** file i holds two positive integers of d_i decimal digits each (a non-zero leading digit followed by uniformly random digits), d = 128, 256, 512, …, 65536, doubling each

time, one integer per line. multiplication_01 begins 70062006841356438916… × 57208186436341040813…, multiplication_05 begins 82756164966649057195… × 95522431381370642844…, and multiplication_10 begins 21402793624087047636… × 42271832072145529502…. Brute force was run for closest pair up to n = 5,000 and schoolbook (digit-by-digit) multiplication up to 16,384 digits; beyond that only the divide-and-conquer result was checked against Python's built-in product. The complete files are in Part2_3/inputs/.

4. Results and Discussion

File	n	Closest distance	Closest pair (x, y) – (x, y)	D&C ms	Brute ms	Speed-up
01	150	31.3124	(2634.437, 4976.024) – (2664.661, 4984.208)	0.7	2.7	3.9×
02	300	28.5058	(3795.432, 9657.979) – (3805.819, 9631.433)	1.2	11.9	9.9×
03	600	13.8641	(1571.420, 3672.085) – (1583.545, 3678.808)	3.3	53.3	16×
04	1200	11.4363	(3736.234, 4282.430) – (3735.920, 4293.862)	9.1	215.0	24×
05	2500	1.8074	(7116.876, 1239.242) – (7117.522, 1237.554)	13.3	751.7	57×
06	5000	3.7950	(809.068, 6847.319) – (811.679, 6844.565)	30.8	2829.0	92×
07	10000	0.9838	(6883.449, 5300.645) – (6882.714, 5301.299)	68.5	—	
08	20000	0.1475	(1867.371, 6968.162) – (1867.350, 6968.308)	164.3	—	
09	40000	0.0785	(8574.537, 1502.610) – (8574.578, 1502.677)	413.0	—	
10	80000	0.1396	(1514.542, 3520.419) – (1514.555, 3520.558)	1233.0	—	

File	Digits	Product digits	Karatsuba ms	Schoolbook ms	Correct
01	128	256	1.2	0.2	✓
02	256	512	3.9	0.3	✓
03	512	1024	11.2	1.4	✓
04	1024	2048	32.8	7.4	✓
05	2048	4096	98.6	44.2	✓
06	4096	8192	303.3	264.8	✓
07	8192	16383	1004.1	2078.0	✓
08	16384	32768	2933.9	10597.0	✓
09	32768	65536	8918.6	—	✓
10	65536	131071	29011.3	—	✓

Correctness. Every closest-pair distance matched brute force to 10^{-9} and every Karatsuba product equalled Python's built-in multiplication, so all twenty benchmark results are correct solutions. **Closest pair.** Doubling n roughly doubles the divide-and-conquer time (e.g. 164 → 413 → 1233 ms for 20k → 40k → 80k, a factor ≈ 2.5–3 that includes the Θ(n log n) sort and Python's set-based partitioning), whereas brute-force time quadruples (215 → 752 → 2829 ms), exactly the $\Theta(n^2)$ prediction; the speed-up grows from 4× at 150 points to 92× at 5,000. The closest distance shrinks as n grows (31.3 → 0.08) because the expected nearest-neighbour distance in a uniform square scales like $1/\sqrt{n}$. **Karatsuba.** For small inputs the schoolbook loop is faster because Python's built-in digit multiplication is implemented in C while Karatsuba's recursion has interpreter overhead per call (14,638 calls already at 256 digits). The curves cross at about 4,000 digits; beyond that Karatsuba's Θ(n^1.585) wins by 2.1× at 8,192 and 3.6× at 16,384 digits, and its own time grows by ≈3× per doubling (1004 → 2934 → 8919 → 29011), matching the three-way recurrence. The 65,536-digit product (131,071 digits) was computed in 29 s. **Interface.** Figure 2 shows the UI mid-run: the current region (dashed), the strip of width 2d around the median line, the strip points and the current best pair, with the explanation panel reporting d_L, d_R and the number of strip pairs checked (always small, confirming the ≤ 7-neighbour bound).

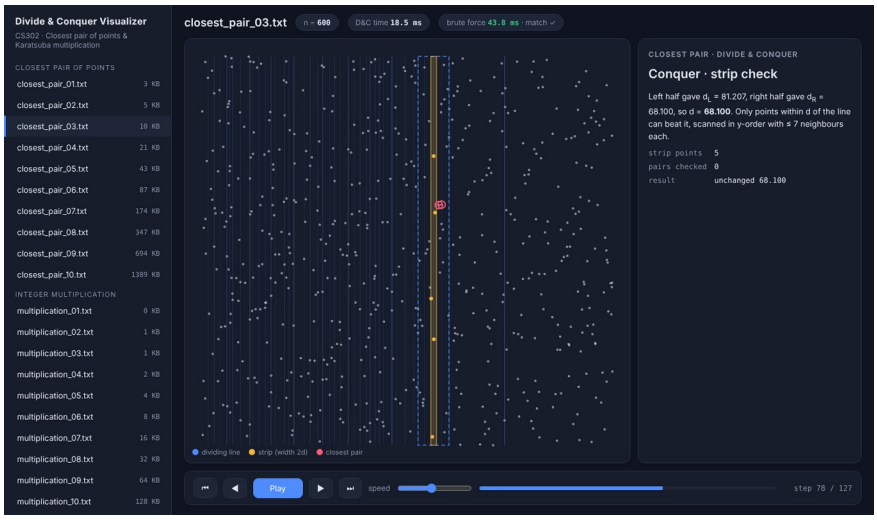


Figure 2. Visual interface during the strip-check step of closest_pair_03.txt (n = 600).

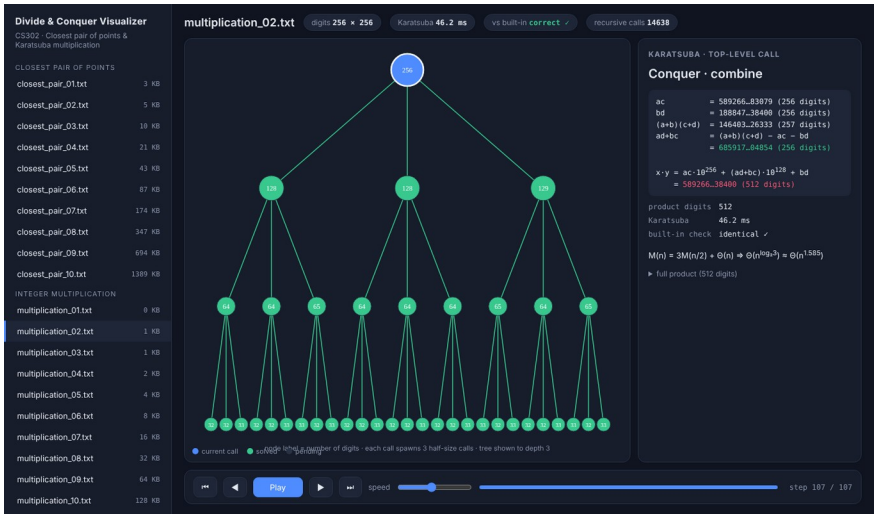


Figure 3. Karatsuba recursion tree (node label = digit count) and the combine step for multiplication_02.txt (256 × 256 digits).

5. Conclusion

Both divide-and-conquer algorithms were implemented, verified on twenty generated datasets and visualised step by step. The experiments confirm the theory: closest pair scales as n log n and beats the Θ(n²) brute force by almost two orders of magnitude at 5,000 points, while Karatsuba, after paying its recursion overhead on small inputs, grows as n^1.585 and is 3.6× faster than schoolbook multiplication at 16,384 digits. Future work could add an iterative base case for Karatsuba (switch to built-in multiplication below ~64 digits) and show a k-d-tree comparison in the interface.

References

[1] A. Levitin, Introduction to the Design and Analysis of Algorithms, 3rd ed., Pearson, 2012, ch. 5.

[2] J. Kleinberg and É. Tardos, Algorithm Design, 1st ed., Pearson, 2006, ch. 5.

[3] A. Karatsuba and Yu. Ofman, “Multiplication of many-digital numbers by automatic computers,” Doklady Akademii Nauk SSSR, vol. 145, pp. 293–294, 1962.

[4] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 3rd ed., MIT Press, 2009, ch. 4 and 33.